

SYSC 3020: Introduction to Software Engineering

Assignment 2: Design (Structure & Interaction) · Lab Walkthrough

SYSC 3020: Introduction to Software Engineering

Assignment 2: Design (Structure & Interaction) · Lab Walkthrough

Fall 2026

TA: Rinkesh Joshi · rinkeshjoshi@cmail.carleton.ca

Lab: LE1 · Wednesday, October 07, 2026 · 2:35 PM to 5:25 PM

Room: Mackenzie Building 4233

Assignment due: Wednesday, October 21, 2026 by 11:59 PM

Weight: 5% of final grade · graded out of 16

The Four Tasks at a Glance

Task A

Use Cases

Turn your A1 SRS into a use case diagram.

- Actor(s)
- Use cases (goals)
- System boundary
- «include» / «extend» only if justified

Output: UC diagram

Task B

Class Diagram

Recover the structure from the code.

- Attributes, operations
- Associations, multiplicities, roles
- Generalization
- Interfaces + realization

Output: Class diagram

Task C

Sequences

Trace ≥ 3 real flows through the code.

- Lifelines, activations
- Message types
- Combined fragments with guards
- A call whose return value is used

Output: ≥ 3 sequence diagrams

Task D

Check & Trace

Prove the models agree with each other + code.

- Consistency check
- Validate vs. code
- Start the RTM in Requirement Yogi

Output: Consistency and Traceability sections

Plus Sprint 2 running across all tasks in Jira (planning, stand-ups, board, review + retro).

Must read: the separate Jira Guide slides: how to post stand-ups and assign roles with labels. Use them for A2 and every assignment after.

What You Submit

Deliverables (one per team, on Brightspace)

1. design.pdf: exported from Confluence
 - Use case diagram (Task A)
 - Class diagram (Task B)
 - ≥ 3 sequence diagrams (Task C)
 - Consistency check + Traceability (Task D)
 - Team & Process (roles, retro, AI Usage & sources note)
2. uml-sources.zip: all your PlantUML `.puml` files
3. Jira board link: shows sprint activity and invite all the TAs so the link opens (see the Jira Guide slides)

List every team member (name + student ID) in the Brightspace submission.

Grading: 4 criteria $\times$ 4 pts = 16 total (Use-case & class model · Interaction model · Consistency & traceability · Process & reporting)

Penalties to Watch

- Missing design.pdf $\rightarrow$ -3 marks
- Missing uml-sources.zip $\rightarrow$ -3 marks
- Late $\rightarrow$ -15% of the full grade per day (accepted up to 48 h late)
- Don't submit: PNGs (they go inside design.pdf) or use case descriptions

Presentation

A random sample of students may be asked to present; it must match what you submitted. Can't explain it $\rightarrow$ affects your individual grade.

Grading Rubric: Quick View

Use-case & Class Model (4 pts)

Both diagrams complete, correct UML notation, and consistent with each other.

Interaction Model (4 pts)

≥ 3 correct sequence diagrams: lifelines, activations, combined fragments, and a genuine cross-object dependency.

Consistency & Traceability (4 pts)

Every lifeline, message, and use case consistent across models (incl. the associations / dependencies behind each message); RTM covers all FRs in the design; models validated against the code.

Process & Reporting (4 pts)

Jira board current with real sprint activity; AI usage disclosed; design report complete and correctly formatted (unrendered diagrams cost marks).

Score Range	Meaning
14-16	Excellent work
10-13	Good work
6-9	Some improvement needed
0-5	Much improvement needed

Each criterion gets 1-4 for assessable work, or 0 if nothing assessable is submitted.

From A1 to A2: What Changes

Reuse from Assignment 1

- SRS: your FRs + user stories are the input to every diagram
- Code Map: your class tables are the starting point for Task B
- Jira + Confluence + Yogi: same workspace, same accounts
- JDK 11 + starter repo: same setup

New or Different

- PlantUML: diagrams-as-code (`.puml`)
- Late penalty is now 15%/day (was 10%)
- Missing file is now -3 (was -2)
- Rotate roles: nobody keeps the same role two sprints running
- Scope adds `game.Game` + `game.SinglePlayerGame`

Note: You do not submit use case descriptions, but write them anyway (lecture template). Their steps become the scenarios for your sequence diagrams in Task C.

Before You Draw › Lecture Notes & Examples

Go through the lecture slides first; use their notation

Lecture	Covers	Use it for
3: Context & Use Case Models	actors, use cases, «include» / «extend», descriptions	Task A
4: Structural Models	classes, associations, multiplicity, aggregation / composition	Task B
5: Behavioural Models	sequence diagrams, messages, combined fragments	Task C

References & examples used in class

1. <https://www.uml-diagrams.org/use-case-diagrams.html>
2. <https://app.diagrams.net>
3. <https://www.uml-diagrams.org/index-examples.html>
4. <https://www.uml-diagrams.org/facebook-authentication-uml-sequence-diagram-example.html>
5. <https://www.uml-diagrams.org/bank-atm-uml-state-machine-diagram-example.html>
6. OMG UML (lecture reference): <http://www.uml.org/>

diagrams.net is fine for sketching, but A2 is PlantUML: every diagram you submit needs its .puml source in `uml-sources.zip`.

Sprint 2 > Run It in Jira

Day 1: Sprint planning

- If Sprint 1 is still open → Backlog → Complete sprint first
- Create the epic: "A2 – Design: Structure & Interaction" (a new epic, same Jira space as A1)
- Break it into tasks → assign each to a member (split a shared task into sub-tasks; see Jira Guide > Jira 4)
- Agree a one-line sprint goal → Create Sprint → Start Sprint
- Rotate roles: Product Owner · Scrum Master · QA Lead

During the sprint

- Stand-ups ≥ 2×/week as Jira comments, three lines per member: done / doing / blocked
- Move tasks Backlog / To Do → In Progress → Done as you work; the board history is graded

Last day: Review + Retrospective

- Demo the deliverable to each other
- Add 3 bullets (keep / change / try) to the Team & Process section of the design page

Confused about Jira? Read the separate Jira Guide slides: board, backlog, sub-tasks, labels for roles, stand-ups, retro, and how to share your board with the TAs.

How You Work > Diagrams-as-Code

 The pipeline (same for every diagram)

```
uc-01.puml —render—► UC-01.png —upload—► Confluence page —export—► design.pdf
|
|_____ zip all .puml _____► uml-sources.zip
```

 Give every diagram a unique name or ID (suggested scheme)

ID	Diagram	File
UC-01	Use case diagram	uc-01.puml
CD-01	Class diagram	cd-01.puml
SD-01 ... SD-03	Sequence diagrams	sd-01.puml ...

The IDs are what your consistency check and RTM point to (Task D).

 Tip: Keep your .puml files in a team folder (e.g., diagrams/), not inside the read-only starter repo.

Setup > Recommended: PlantUML Website (No Install)

 The preferred way: nothing to install

1. Write each diagram in a plain-text editor (Notepad, TextEdit, VS Code) and save it as a `.puml` file (e.g., `sd-01.puml`)
2. Copy the text into the official PlantUML web server: <https://www.plantuml.com/plantuml/uml/> → Submit
3. Download the PNG → upload it to your Confluence design page
4. To change a diagram: edit the `.puml` file, paste it again, download again

The site can't save `.puml` files, so your `.puml` files are the ones that go in `uml-sources.zip`.

 Windows: Notepad

File → Save as → Save as type: All files (*.*) → file name `sd-01.puml` → Save

(Otherwise Notepad may save `sd-01.puml.txt`.)

 macOS: TextEdit

Format → Make Plain Text first, then save as `sd-01.puml` → if asked, choose Use `.puml` (not `.txt`)

Prefer to work offline? Installing PlantUML is optional: next slides.

Setup › Optional: Install PlantUML Locally

PlantUML runs on Java: your JDK 11 from A1 is enough. Use case + class diagrams also need Graphviz (sequence diagrams don't).

Windows

- Download plantuml.jar from:
<https://plantuml.com/download>
- Put it next to your `diagrams/` folder
- Graphviz is bundled in the jar on Windows. If `-testdot` fails:

```
# Install Graphviz manually  
winget install Graphviz.Graphviz
```

macOS

```
# Homebrew (recommended)  
# (also installs Graphviz)  
brew install plantuml
```

- Or download plantuml.jar (same link) and install Graphviz separately:

```
brew install graphviz
```

No Graphviz at all? Add `!pragma layout smetana` as the 2nd line of a `.puml` file.

Setup › Verify & Render

Windows

```
# Check PlantUML + Graphviz
java -jar plantuml.jar -testdot

# Expected: "Installation seems OK.
#   File generation OK"

# Render every .puml in a folder
java -jar plantuml.jar -tpng diagrams
```

macOS

```
# Check PlantUML + Graphviz
plantuml -testdot

# Expected: "Installation seems OK.
#   File generation OK"

# Render every .puml in a folder
plantuml -tpng diagrams
```

PNGs appear next to the .puml files, named after @startuml <name> (e.g., @startuml SD-01 → SD-01.png).

Tip: Text blurry in the exported PDF? Add skinparam dpi 200 to the .puml and re-render.

Setup › Live Preview & Zipping

Optional: preview while you type (VS Code)

1. Extensions → install PlantUML (by jebbs)
2. Open a `.puml` file → Alt + D (Windows) / Option + D (macOS)
3. Export: Command Palette → "PlantUML: Export Current Diagram"

Build `uml-sources.zip`

Windows

```
# PowerShell
Compress-Archive `
  -Path diagrams\*.puml `
  -DestinationPath uml-sources.zip
```

macOS

```
zip uml-sources.zip diagrams/*.puml
```

Check: every diagram in `design.pdf` has its `.puml` in the zip.

Task A > What You Actually Do

Step 1: Pick what you will model from your SRS

Go through your functional requirements (B3) and user stories (B4). Group them by the goal the actor wants to achieve.

Step 2: Draw the use case diagram

- Actor(s): stick figure outside the boundary; singular noun naming a role (e.g., Player)
- Use cases: ellipses named with verb phrases = the actor's goal (e.g., Play a level); each should pass the "one person, one sitting" test (Lecture 3)
- System boundary: a rectangle labelled with the system name (JPacman)
- Associations: plain lines between actor and use case (no arrowheads)

Step 3: Trace each use case

Every use case → at least one FR or user story in your SRS, and to `doc/scenarios.md` where it applies (Stories 1-4). Otherwise, cite the code.

Task A › Actors, «include» & «extend»

Who is an actor?

- Actor = an entity outside the JPacman boundary (e.g., the Player)
- `Ghost`, `Level`, `Board` are internal classes → not actors
- "As a ghost..." in `doc/scenarios.md` is a story role, not a UML actor; ghost behaviour happens inside the boundary

«include» vs «extend»: only when genuinely warranted

	«include»	«extend»
Meaning	Base always does it	Adds optional / conditional behaviour
Arrow	Base ..► included	Extension ..► base
Base runs alone?	No, always triggers it	Yes, independent of it

If you can't name the condition for an «extend», it isn't one.

`scenarios.md` S2.5 says it "extends S2.1". That's a scenario note; decide whether it's a genuine UML «extend».

Red flags

- Use cases named after methods (`Level.move`) or UI clicks
- Ghosts or game classes drawn as actors
- Every use case wired with «include» "just because"

Task A › PlantUML Use Case Syntax

```
@startuml UC-01
left to right direction
actor Player

rectangle "JPacman" {
    usecase "Play a level" as play
    usecase "Use case B" as ucB
    usecase "Use case C" as ucC
}

Player -- play
play ..> ucB : <<include>>
ucC ..> play : <<extend>>
@enduml
```

- `rectangle "JPacman" { ... }` → the labelled system boundary
- `--` → actor to use case association (no arrowhead)
- `..> + <<include>> / <<extend>>` → watch the arrow direction

(Use cases B and C are placeholders; derive yours from your SRS.)

Task A > Use Case Descriptions (Lecture 3 Template)

Not submitted, but you need them

Each basic flow is the scenario your sequence diagram traces in Task C.

Template fields

Use Case Name · Brief Description · Primary Actor · Secondary Actor · Precondition · Dependency («include» / «extend») · Generalization · Basic Flow (steps + postcondition) · Specific / Global / Bounded Alternative Flows · Special Requirements

Restricted natural language

INCLUDE USE CASE · EXTENDED BY USE CASE · RFS · IF ... THEN ... ENDIF · VALIDATES THAT · DO ... UNTIL · RESUME STEP · ABORT

Example, checked against the code

```
Use Case Name: Start Game
Primary Actor: Player
Precondition: The game window is open; the game is not in progress.
Basic Flow:
1. The Player presses Start.
2. The system VALIDATES THAT the player is alive and pellets remain.
3. The system starts moving the ghosts.

Postcondition: the game is in progress.

Specific Alternative Flow: RFS Basic Flow 2 (player dead or no pellets left)
1. ABORT.

Postcondition: the game stays stopped.
```

(Source: `Game.start` → `Level.start`; `doc/scenarios.md` S1.1)

Task B > What's In Scope

Packages to cover

`board · level · npc (+ npc.ghost) · points · plus game.Game and game.SinglePlayerGame`

Start from Board, Unit (+ subtypes), Level, PointCalculator → find the rest in `src/`.

What you may leave out

- Factories, parsers, loaders (`BoardFactory`, `MapParser`, `GhostFactory`, ...)
- Private nested classes (e.g., `Level.NpcMoveTask`)
- Out-of-scope types (`Sprite`, `List`, ...) → name only, no internals

Exception: if a class you could omit appears in one of your sequence diagrams, it must be in the class diagram with its relevant operations.

Find every extends / implements fast

```
cd src/main/java/nl/tudelft/jpacman

# macOS
grep -rE "(class|interface) .* (extends|implements)" board level npc points game

# Windows (cmd)
findstr /s /n /r /c:"class .* extends" /c:"class .* implements" /c:"interface .* extends" *
.java
```

Task B > Attribute or Association?

The rule (from lecture)

Attribute types are primitives, data types, or enumerations only: no other classes as types. UML primitives: Boolean · Integer · Real · String (Lecture 4). A field whose type is another class → draw an association with a role name + multiplicity (omitted multiplicity = 1).

Field in the code	How you draw it
<code>private int score; (Player)</code>	Attribute: - <code>score</code> : Integer
<code>private Direction direction; (Unit)</code>	Attribute: - <code>direction</code> : Direction (enum)
<code>private final Board board; (Level)</code>	Association Level → Board, role <code>board</code> , mult. 1
<code>List<Unit> ..., Map<..., Square> ...</code>	Association, role name, 0..* (or what the code allows)

Visibility

+ public · - private · # protected · ~ package-private

 Hint: The field's declaring class sets the navigability: the arrow points away from the class that holds the field. If both sides hold a field, it's bidirectional.

Task B > Relationships Need Evidence

Read straight from the code

- Generalization: `extends` → solid line, hollow triangle (e.g., several units extend Unit)
- Realization: `implements` → dashed line, hollow triangle (e.g., `PointCalculator`, `CollisionMap`)
- Interface: class box with «interface»
- Dependency: dashed arrow when a class only uses another (parameter, local var, static call) with no field

Must be justified

Aggregation / composition: not just because a field exists. Show evidence:

- Who creates the part?
- Does the part die with the whole?
- Can the part be shared by others?

Aggregation (hollow $\diamond$ on the whole): the part can exist without the whole. Composition (filled $\blacklozenge$ on the whole): the part cannot (existence dependency).

Write the evidence next to the relationship in your report.

No evidence → plain association.

Hint: Interfaces hide in odd places; check for nested interfaces declared inside classes too, and who implements them.

Task B › PlantUML Class Syntax

```
@startuml CD-01
' show + - # instead of icons
skinparam classAttributeIconSize 0
hide circle
hide empty members

abstract class Unit {
- direction : Direction
+ getSquare() : Square
+ occupy(target : Square) : void
{abstract} + getSprite() : Sprite
}
class Player {
- score : Integer
+ addPoints(points : Integer) : void
}
interface PointCalculator <<interface>> {
+ consumedAPellet(p : Player, pel : Pellet) : void
}
enum Direction <<enumeration>>

' generalization · realization · association
(mult. + role)
Unit <|-- Player
PointCalculator <|.. DefaultPointCalculator
Level --> "1\n-board" Board
@enduml
```

Relationship arrows

Relationship	PlantUML
Generalization	Parent < -- Child
Realization	Iface < .. Impl
Association	A --> "1\n-role" B
Aggregation (◇ whole)	Whole o-- Part
Composition (◆ whole)	Whole *-- Part
Dependency	User ..> Used

Multiplicity at both ends: A "1" --> "0..*\n-items" B

Task C › What Every Diagram Needs

Requirements (from the handout)

- ≥ 3 sequence diagrams, each a real event flow in JPacman (input / tick $\rightarrow$ Level / units / collaborators $\rightarrow$ outcome)
- Every call message = a real method on the target class (replies + actor/system events exempt)
- ≥ 1 diagram uses combined fragments with guards (`alt`, `opt`, `loop`; Lecture 5 also covers `par`, `break`, `ref`)
- ≥ 1 diagram shows a cross-object dependency: A calls B, gets a value back, uses it
- May start at a core entry method: `Game.move`, `Game.start`, `Game.stop`
- UI event-handling internals: not needed

Message types $\rightarrow$ PlantUML

Type	UML (Lecture 5)	PlantUML
Synchronous	solid line, filled head	<code>a -> b : op(args)</code>
Asynchronous	solid line, open head	<code>a ->> b : op(args)</code>
Reply	dashed line, or label the call <code>x = op(args)</code> (preferred)	<code>b -->> a : x</code> · <code>a -> b : x = op(args)</code>
Create	dashed, open head, «create»	<code>create b then a -->> b : <<create>></code>
Activation	box on the lifeline	<code>a -> b ++ : op() ... deactivate b</code>

Instances are named `o : Object` (or `: Object`), e.g. `level : Level`.

Task C › Trace a Flow from the Code

Step 1: Open the entry method and follow every call

For each call, write down: receiver object · method · arguments · what's returned · is it used?

Step 2: Map the code shape to UML

In the code	In the diagram
<code>if / else</code>	<code>alt</code> with a guard per branch
<code>if</code> with no <code>else</code>	<code>opt</code> with a guard
<code>for</code> / repeated tick	<code>loop</code> with a guard
<code>new X(...)</code>	create message
call to a private helper	self-call on the same lifeline

Step 3: Send each message to the object that actually receives it

- e.g., `getSquareAt` is called on the unit's current `Square`, not on `Level`
- The lifeline is the runtime object: the game is a `SinglePlayerGame` (an abstract type like `Square` is fine when the concrete class is private)
- Check which `CollisionMap` (→ `LevelFactory`) and which `PointCalculator` (→ `PointCalculatorLoader` + `scorecalc.properties`) the running game uses

Hint: Each ghost runs on its own timer thread (one `ScheduledExecutorService` per ghost in `Level`; `NpcMoveTask` reschedules itself). Think about which messages there are asynchronous.

Task C › PlantUML Sequence Syntax

A starter for the handout's example, Player eats a pellet:

```
@startuml SD-01
actor Player as user
participant "game : SinglePlayerGame" as game
participant "level : Level" as level
participant "location : Square" as loc

user -> game ++ : move(player, direction)
' TODO: what does Game.move check first?
game -> level ++ : move(player, direction)
' TODO: how does level get the player's current square?
level -> loc ++ : destination = getSquareAt(direction)
deactivate loc

alt pellet found
' TODO: trace the rest from the code
else no pellet
' TODO
end
@enduml
```

- ++ opens an activation, deactivate closes it (return x would draw a reply arrow instead)
- destination = getSquareAt(...): the lecture's preferred way to show a returned value
- Write guards without brackets (alt pellet found); PlantUML adds the [] itself
- The TODOs mark calls this starter skips: verify every message against the code

Task C › Choosing Your Three

Diagram 1: Player eats a pellet (from the handout)

Allowed as one of your three, but trace it yourself from `Game.move`.

Diagrams 2 & 3: Pick from your SRS

Good candidates (each maps to a story in `doc/scenarios.md`):

- Start / stop the game (Stories 1 & 4) → from `Game.start` / `Game.stop`
- A ghost moves on a tick (Story 3) → internally triggered: actor lifeline optional, but say which use case it supports
- The player dies (S2.4 player moves into a ghost · S3.4 ghost moves into the player)

Before you commit to a scenario

- Does it trace to ≥ 1 FR or user story in your SRS?
- Does it realise a use case in your UC diagram?
- Across your three: combined fragment ✓ · cross-object dependency ✓

Task D › Consistency Check

Three checks: document each one

1. Lifelines: every software lifeline is an instance of a class (or implements an interface) in CD-01; every actor lifeline is an actor in UC-01
2. Messages: every call maps to an operation declared, inherited, or specified by an interface of the receiver; sender + receiver are linked by an association or dependency in CD-01
3. Scenarios: each SD realises a scenario of a use case in UC-01; internally triggered flows explain how they support the use case

Suggested table format (one row per message)

Diagram · message	Receiver	Operation (cite)	Link in CD-01
SD-01 · <code>move(player, direction)</code> ...one row per call message...	<code>level : Level</code>	<code>Level.move(Unit, Direction)</code>	SinglePlayerGame → Level

A message with no association or dependency behind it → fix the class diagram.

Task D > Validate Against the Code

Put all validation evidence in design.pdf (handout: "cite the evidence in your report").

Class diagram

- IntelliJ IDEA (free student licence: jetbrains.com/student) → right-click a package → Diagrams → Show Diagram
- Reconcile with yours: note and explain every difference (e.g., the tool shows `board` as a field; you drew it as an association)
- No tool? → a short table comparing your diagram with the source files

Use case diagram

Each use case → an FR or user story in your SRS, plus `doc/scenarios.md` where it applies; otherwise cite the source code.

Sequence diagram: validate one flow, two options

- Debugger: step through it and record the calls in execution order (next slide)
- Test: cite a test in `src/test/java` or `src/default-test/java` that exercises it; state which calls it verifies; confirm the rest in the debugger or source

Task D › Step Through a Flow

Option 1: Debugger (IntelliJ)

1. Open the repo as a Gradle project → set a breakpoint at your entry method (e.g., `Game.move`)
2. Run → Debug `nl.tudelft.jpacman.Launcher` → press Start, then trigger the event (e.g., an arrow key)
3. Step Into (F7) / Step Over (F8) → record each call in order (a call-stack screenshot helps)

Option 2: Cite an existing test

Run `./gradlew test` (macOS / PowerShell) or `gradlew test` (Windows cmd); it runs both test folders.

Test (in <code>src/default-test/java/.../jpacman/</code>)	Exercises
<code>LauncherSmokeTest</code>	<code>Game.start · move (+10 per pellet) · death · stop</code>
<code>integration/StartupSystemTest</code>	<code>Launcher.launch · Game.start</code>
<code>level/LevelTest</code>	<code>Level start / stop / registerPlayer</code>
<code>npc/ghost/NavigationTest</code>	<code>Navigation.shortestPath / findNearest</code>

Tests assert outcomes (score, alive, in-progress) or use mocks (LevelTest); none checks call order. State which calls the test verifies, then confirm the order in the debugger or source.

Task D › Start the RTM in Requirement Yogi

Step 1: Link requirements to diagrams

- On the design page, each diagram sits in its own section with its ID (UC-01, CD-01, SD-01 ...)
- In each section, reference the FR keys that diagram represents (e.g., SJC-001) → Yogi records where each requirement is cited
- Link NFRs too, where applicable

Step 2: Open and export the traceability view

1. Open Requirement Yogi → Traceability (in Yogi Cloud: Requirements in the Confluence space sidebar)
2. Add columns → Links (pages where each requirement is cited) + Description
3. Export to Excel (top right), or take a screenshot
4. Put it in a Traceability section of the design page

Fallback: Yogi links not working? Include a manual table: requirement ID → diagram ID(s). Every FR in your design must appear. (A5-A6 will extend this with tests.)

The Design Page in Confluence

Confluence → Create → Blank page, e.g., "Design – JPacman Core"

Sections

1. Use case diagram: UC-01
2. Class diagram: CD-01 with aggregation / composition evidence
3. Sequence diagrams: SD-01 ... SD-03 (each with its FR / story + use case)
4. Consistency check + model validation
5. Traceability: Yogi view or manual RTM
6. Team & Process: roles, retro (keep / change / try), AI Usage & sources note

Embedding diagrams

- Type `/image` → upload the rendered PNG
- One image per diagram, captioned with its ID
- Diagrams must be rendered: no raw PlantUML text in place of a picture

Export

Page → ... → Export → PDF → rename to design.pdf

Self-Check Before You Submit

- ☐ UC-01: actor(s), use cases, labelled system boundary, derived from the SRS
- ☐ CD-01: attributes, operations, associations, multiplicities, roles, generalization, interfaces + realization, in correct notation
- ☐ ≥ 3 SDs tracing real flows with correct notation; every call message is a real method
- ☐ ≥ 1 combined fragment $\cdot \geq 1$ cross-object dependency
- ☐ Consistency check with code references; Traceability section in design.pdf
- ☐ Models validated: class diagram reconciled (or comparison table) $\cdot$ one flow checked $\cdot$ use cases traced
- ☐ uml-sources.zip has every .puml; Team & Process section (roles, retro, AI Usage & sources note)
- ☐ Jira board: planning, assigned tasks, stand-up comments; all TAs invited
- ☐ Submitted on Brightspace: design.pdf + uml-sources.zip + board link; all members' names + student IDs listed on Brightspace

Remember: "Work that is not clearly explained or not traceable to the actual code will not be considered correct."
Any team member may also be asked to present it.

